

1. The Basic Assertion Goal: Understand the simplicity of pytest syntax. Write a file named `test\_logic.py`. Create a test function `test\_math\_operations` that uses simple `assert` statements to verify: That 15×3 equals 45 . That the string `pytest` is present within the phrase `Learning pytest is fun`.

```
# test_logic.py

def test_math_operations():

    assert 15 * 3 == 45

    assert "pytest" in "Learning pytest is fun"
```

2. Using Fixtures for Setup Goal: Replace `setUp` and `tearDown` with the modern `@pytest.fixture` approach. Create a pytest fixture named `sample\_dict` that returns a dictionary: `{"name": "Alice", "role": "Dev"}`. Write a test function `test\_dict\_keys` that: Accepts the fixture as an argument. Asserts that the key `role` exists in the dictionary. Asserts that the value of `name` is `Alice`.

```
import pytest

@pytest.fixture
def sample_dict():

    return {"name": "Alice", "role": "Dev"}

def test_dict_keys(sample_dict):

    # Check if "role" key exists

    assert "role" in sample_dict
```

```
# Check if name value is Alice
assert sample_dict["name"] == "Alice"
```

3. Handling Exceptions Goal: Use the `pytest.raises` context manager. Write a function `get_element(my_list, index)` that returns an item from a list. Write a test function `test_index_error` that uses `with pytest.raises(IndexError):` to verify that attempting to access index `10` of the list `[1, 2, 3]` correctly triggers an error.

```
import pytest

def get_element(my_list, index):
    return my_list[index]

def test_index_error():
    with pytest.raises(IndexError):
        get_element([1, 2, 3], 10)
```

4. Parameterized Testing Goal: Run the same test logic with multiple sets of data. Use the `@pytest.mark.parametrize` decorator to create a single test function `test_is_even`. Pass three different inputs to the test: `2`, `10`, and `22`. The test should assert that each input `% 2 == 0`. Observe how pytest treats these as three separate test cases in the output.

```
import pytest

@pytest.mark.parametrize("number", [2, 10, 22])
def test_is_even(number):
    assert number % 2 == 0
```

5. Cleaning Up with Fixture Yields Goal: Manage resources (like files or database connections) properly. Create a fixture named `temp\_file`. In the Setup phase : Create a new text file named `test.txt` and write ``Hello World`` into it. Yield the filename to the test. In the Teardown phase (after the yield): Use the `os` module to delete the file. Write a test `test\_file\_content` that reads the file and verifies the text matches.

```
import pytest
import os
@pytest.fixture
def temp_file():
    # Setup phase
    filename = "test.txt"
    with open(filename, "w") as file:
        file.write("Hello World")
    # Give filename to test
    yield filename
    # Teardown phase
    os.remove(filename)
def test_file_content(temp_file):
    with open(temp_file, "r") as file:
        content = file.read()
    assert content == "Hello World"
```